

PART 11

Maschinensprache

Maschinensprache

- Computer werden durch Programme gesteuert, die aus Befehlen in Maschinensprache bestehen
- Diese Befehle liegen im Hauptspeicher und werden nacheinander abgearbeitet
- Die Maschinensprache wird vom Hersteller einer CPU definiert und kann vom Programmierer nicht verändert werden
- Der Hersteller legt fest, welcher Zahlencode im Speicher welchem Maschinenbefehl entspricht

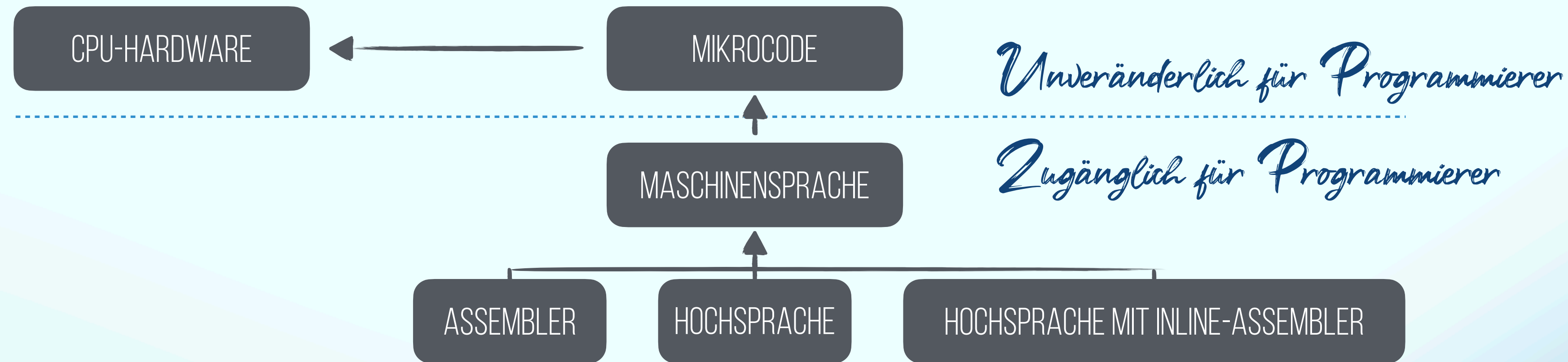
Adresse:	
0x00B813A0	55 8b ec 81 ec c0 00 00 00 53 56 57 8d bd 40 ff ff ff b9 30 00 00
0x00B813A1	00 b8 cc cc cc cc f3 ab 8b f4 68 3c 57 b8 00 ff 15 bc 82 b8 00 83
0x00B813A2	c4 04 3b f4 e8 66 fd ff ff 33 c0 5f 5e 5b 81 c4 c0 00 00 00 3b ec
0x00B813A3	e8 54 fd ff ff 8b e5 5d c3 cc cc cc cc cc cc cc cc cc cc cc cc
0x00B813A4	cc cc cc cc cc cc ff 25 bc 82 b8 00 cc cc cc cc cc cc cc cc cc
0x00B813A5	cc cc 75 01 c3 55 8b ec 83 ec 00 50 52 53 56 57 8b 45 04 6a 00 50
0x00B813A6	e8 80 fd ff ff 83 c4 08 5f 5e 5b 5a 58 8b e5 5d c3 cc cc cc cc cc
0x00B813A7	cc cc cc cc cc cc 8b ff 55 8b ec 51 53 56 57 33 ff 8b f2 39 3e 8b
Bedeutung: (Prozessortyp abhängig)	
00B813A0	push ebp
00B813A1	mov ebp, esp
00B813A2	sub esp, 0C0h
00B813A3	push ebx
00B813A4	push esi
00B813A5	push edi

- Damit ist ein Programm in Maschinensprache immer nur auf einer bestimmten CPU bzw. auf den untereinander kompatiblen CPUs einer Prozessorfamilie lauffähig

Maschinensprache / Assembler

- Eine Maschinensprache ist die Menge von Befehlen, die einem Programmierer für eine konkrete CPU zur Verfügung steht
- Diese Befehle werden normalerweise in der Praxis nicht direkt durch ihren Zahlencode eingegeben, sondern in einer Assemblersprache
- Eine Assemblersprache ist eine lesbare Art, Programme in Maschinensprache zu formulieren
- So wird z.B. für einen Sprungbefehl in der Assemblersprache **jmp** verwendet (für engl. „jump“)
- Es gibt somit eine eindeutige Abbildung zwischen Maschinensprache und Assemblersprache:
 - Ein **Assembler** ist ein Übersetzer von Assemblersprache in Maschinensprache
 - Ein **Disassembler** ist ein Übersetzer von Maschinensprache in Assemblersprache

Erzeugung von Maschinensprache



- Softwareentwicklung findet heutzutage typischerweise in einer höheren Programmiersprache statt (z.B. C/C++, Java,...)
- Um ein in einer höheren Programmiersprache geschriebenes Programm ausführen zu können, muss dies jedoch letztendlich durch einen Compiler in eine äquivalente Folge von Maschinenbefehlen übersetzt werden
- Beim Kompilieren kann angegeben werden, für welchen CPU-Typ der Maschinencode erzeugt werden soll
- Daher kann ein Programm in Hochsprachen im Gegensatz zu einem Assemblerprogramm leicht auf andere Hardware angepasst werden

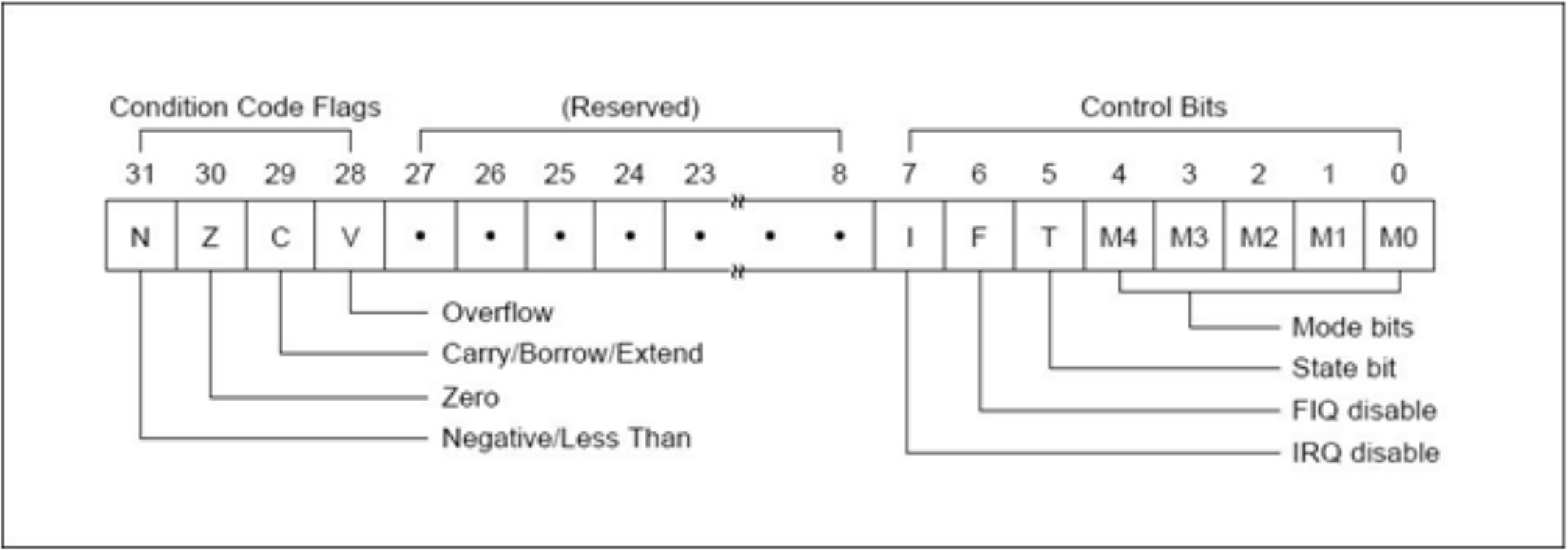
ARM CORTEX-M Register

- R0
- R1
- R2
- R3
- R4
- R5
- R6
- R7
- R8
- R9
- R10
- R11
- R12
- R13 (SP)
- R14 (LR)
- R15 (PC)
- xPSR

Low register

High register

Program Status Register



Speicherorganisation

Byteadressierbarer Speicher: Byte / Halfword / Word / Doubleword

Lade **Byte** von Adresse 4 nach R0:

0x 0000 0014

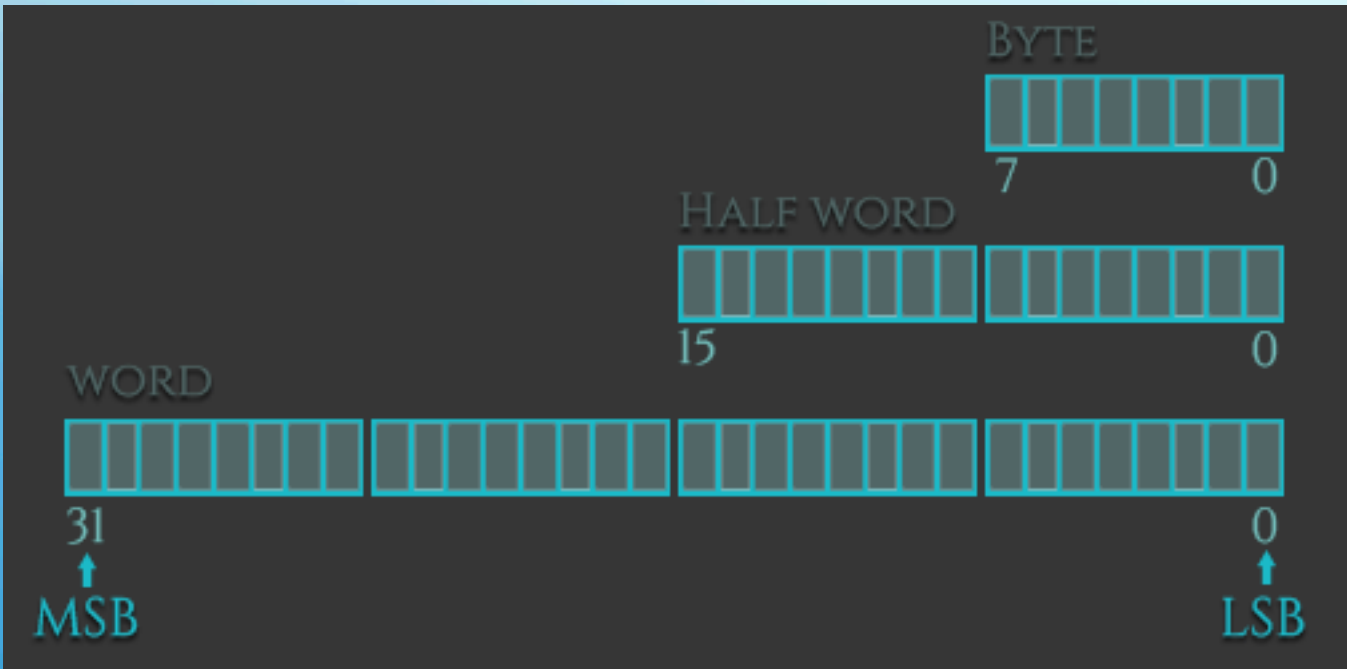
Lade **Halfword** von Adresse 4 nach R0:

0x 0000 1514 little endian
0x 0000 1415 big endian

Lade **Word** von Adresse 4 nach R0:

0x 1514 1312 little endian

ADRESSE	INHALT
0	0x10
1	0x11
2	0x12
3	0x13
4	0x14
5	0x15
6	0x16
7	0x17
8	0x18



Format von Assemblerbefehlen

- In Assembler wird je ein Befehl pro Zeile geschrieben
- Dabei hat ein Assemblerbefehl die Form: [Label] [Operation] [Operand] [;Kommentar]
- Die eckigen Klammern sollen andeuten, dass diese Komponente optional ist, d.h. es ist auch eine leere Zeile möglich
- Zwischen Groß- und Kleinschreibung wird nicht unterschieden (außer wenn im Inline-Assembler auf C/C++ Variablen zugegriffen wird)
- Leerzeichen zwischen den Komponenten werden ignoriert; Bei einem Semikolon beginnt ein Kommentar, der sich bis zum Zeilenende erstreckt

```
...  
Init MOV R0,#150 ;R0=150  
...
```

Format von Assemblerbefehlen

- Viele Assemblerbefehle haben die Form: `op ziel, quelle`
- Dies bedeutet, dass die Operanden Ziel und Quelle mit der Operation **op** verknüpft werden und das Ergebnis in **ziel** gespeichert
- ziel** kann ein Register oder eine Speicheradresse sein
- quelle** kann ein Register, eine Speicheradresse oder ein konstanter Zahlenwert sein

```
ldr = Load Word
ldrh = Load unsigned Half Word
ldrsh = Load signed Half Word
ldrb = Load unsigned Byte
ldrshb = Load signed Bytes

str = Store Word
strh = Store unsigned Half Word
strsh = Store signed Half Word
strb = Store unsigned Byte
strshb = Store signed Byte
```

Instruction	Description	Instruction	Description
MOV	Move data	EOR	Bitwise XOR
MVN	Move and negate	LDR	Load
ADD	Addition	STR	Store
SUB	Subtraction	LDM	Load Multiple
MUL	Multiplication	STM	Store Multiple
LSL	Logical Shift Left	PUSH	Push on Stack
LSR	Logical Shift Right	POP	Pop off Stack
ASR	Arithmetic Shift Right	B	Branch
ROR	Rotate Right	BL	Branch with Link
CMP	Compare	BX	Branch and eXchange
AND	Bitwise AND	BLX	Branch with Link and eXchange
ORR	Bitwise OR	SWI/SVC	System Call

GENERAL PURPOSE REGISTERS

REGISTERS

R0 - R6	General purpose	
R7	Syscall number	
R8 - R10	General purpose	
R11	Frame Pointer	FP
R12	Intra Procedural	IP
R13	Stack Pointer	SP
R14	Link Register	LR
R15	Program Counter	PC

General purpose registers used for temporary values. R7 stores syscall, used for syscall invocation.

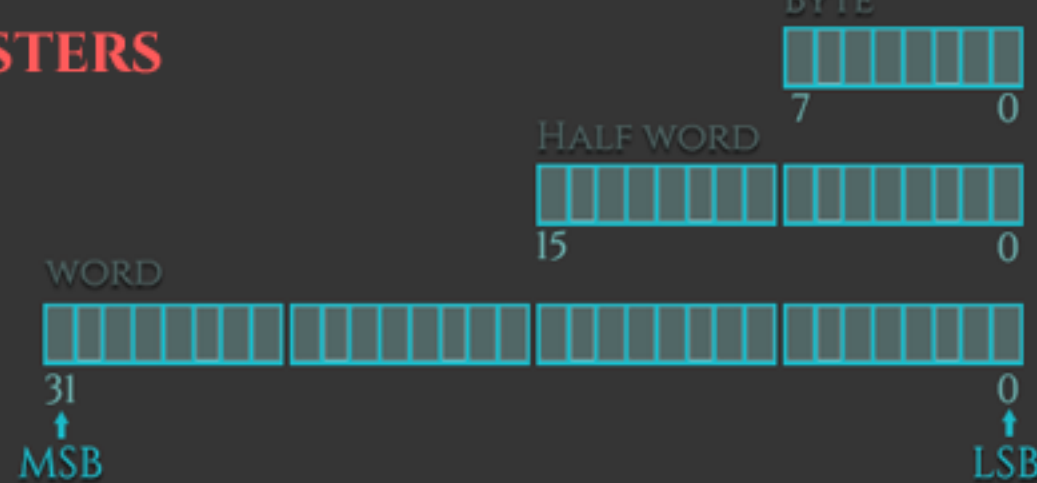
Points to bottom of the Stack Frame, keeps track of boundaries on the stack.

Intra Procedure call scratch Register

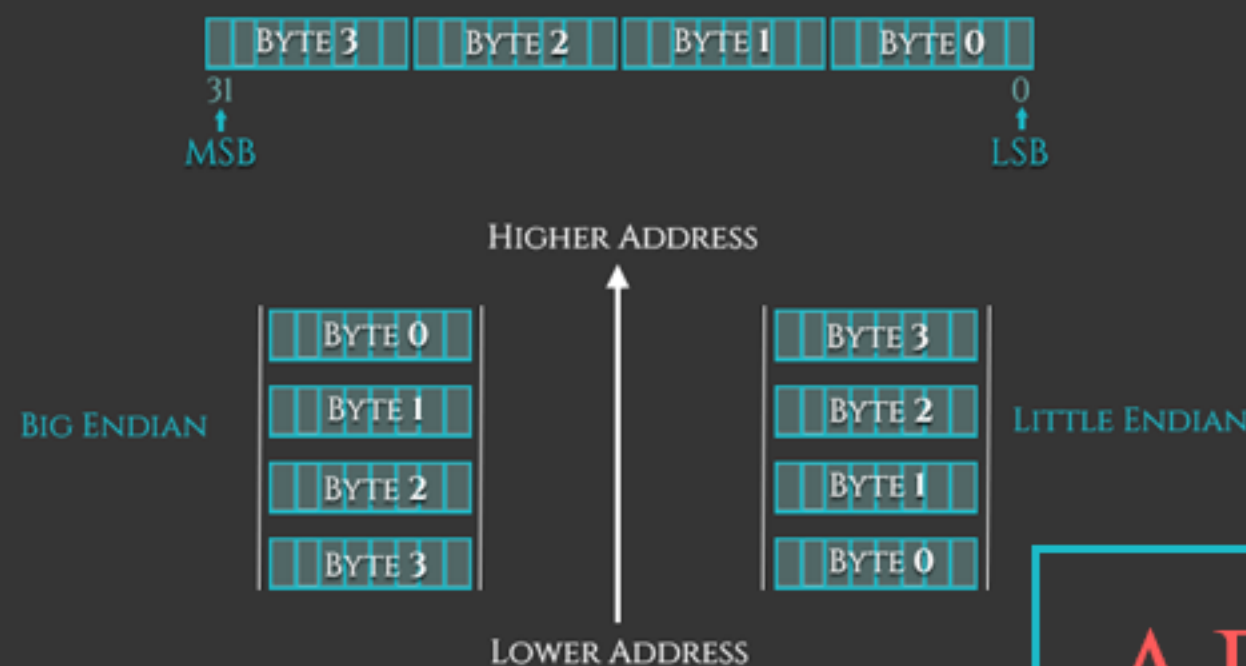
Points to top of the Stack. Used for allocating space on the Stack.

Receives the return address when a BL or BLX instruction is executed.

Holds the address of the next instruction to be executed.



ENDIANNESS



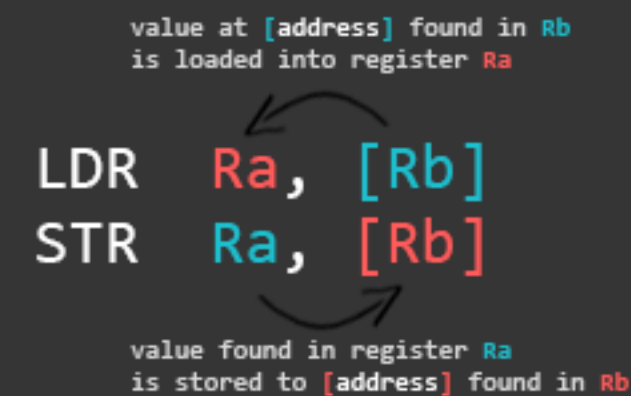
ARM

32-BIT

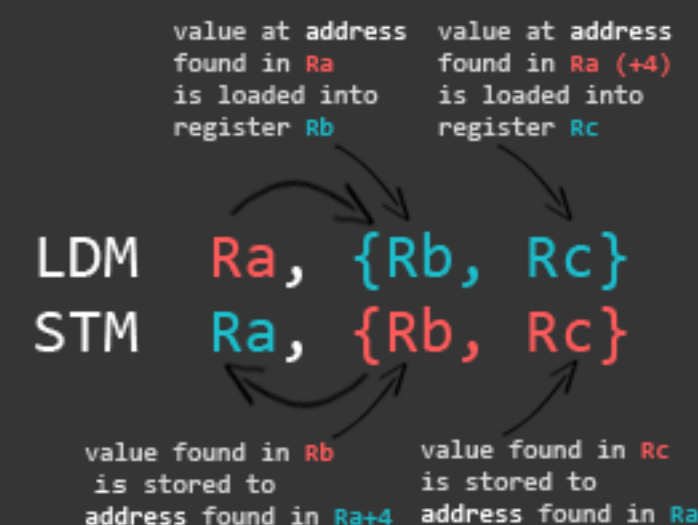
CONDITIONAL EXECUTION

Condition Code	Meaning	Flags Tested
EQ	Equal (==)	Z == 1
NE	Not Equal (!=)	Z == 0
GT	Signed >	(Z==0) && (N==V)
LT	Signed <	N != V
GE	Signed >=	N == V
LE	Signed <=	(Z==1) (N!=V)
CS or HS	U. Higher or Same	C == 1
CC or LO	U. Lower	C == 0
MI	Negative -	N == 1
PL	Positive +	N == 0
AL	Always executed	-
NV	Never executed	-
VS	S. Overflow	V == 1
VC	No Overflow	V == 0
HI	U. Higher	(C==1) && (Z==0)
LS	U. Lower or same	(C==0) (Z==0)

LOAD AND STORE



LOAD AND STORE MULTIPLE



Given:
r0 = 1
r1 = 2
r2 = 3

INSTRUCTION	EXAMPLE	RESULT
MOV	mov r3, #3	r3 = 3
ADD	add r3, r0, r0	r3 = 1 + 1
SUB	sub r3, r0, r0	r3 = 1 - 1
MUL	mul r3, r0, r0	r3 = 1 * 1
LSL	lsl r3, r0, #2	r3 = 1 << 2 = 4
LSR	lsr r3, r0, #2	r3 = 1 >> 2 = 0
ASR	asr r3, r0, #2	r3 = 1 asr 2 = 0
ROR	ror r3, r0, #2	r3 = 0x40000000
AND	and r3, r1, r0	r3 = 1 and 2 = 0
ORR	orr r3, r1, r0	r3 = 1 orr 2 = 3
EOR	eor r3, r1, r0	r3 = 1 xor 2 = 3

ARM INSTRUCTIONS

All instructions conditional
32-bit instructions

THUMB INSTRUCTIONS

No conditional instructions
16-bit instructions

CPSR / APSR

(Current Program Status Register)

N	Z	C	V	Q	J	GE	E	A	I	F	T	M
---	---	---	---	---	---	----	---	---	---	---	---	---

Cmp/Test Instructions

CMP (compare),
CMN (compare negative),
TEQ (test equivalence),
TST (test bits)

always update flags.
update flags only if S suffix set unchanged.

Other instructions, like
MOVS (move, update flag)
ADDS (add, update flag)
SUBS (subtract, update flag)
[...]

Example: CMP & LT

```
mov r0, #2
mov r1, #4
cmp r0, r1
movlt r2, #4
movgt r2, #8
```

r0 = 2
r1 = 4
r0 - r1 = 2 - 4 = -2
(N != V) == true
(N == V) == false

N	Z	C	V	Q	[...]
1	0	0	0	0	

set negative flag

Example: CMP & EQ

```
mov r0, #2
mov r1, #2
cmp r0, r1
beq func1
mov r2, #8
```

r0 = 2
r1 = 2
r0 - r1 = 2 - 2 = 0
(Z == 1) == true

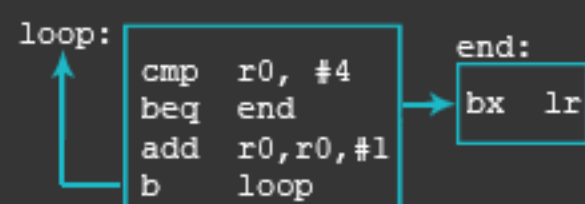
N	Z	C	V	Q	[...]
0	1	1	0	0	

set zero flag

BRANCH (B)

SYNTAX

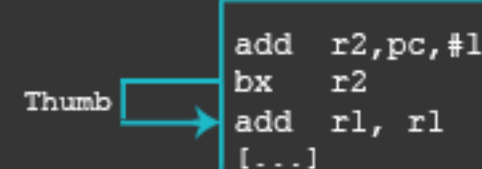
b[cond] label
b label



BRANCH & EXCHANGE (BX)

SYNTAX

bx[cond] Rm
bx Rm



SWITCH TO THUMB MODE

- Set LSB of next instruction to 1 and move it to register
- Branch to R2

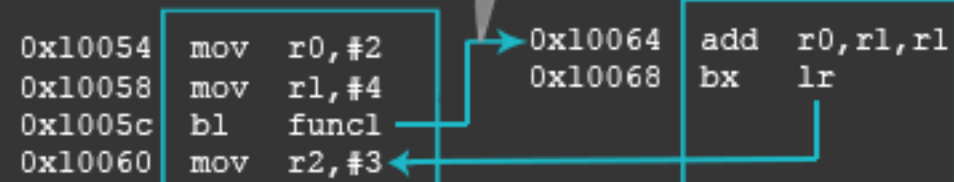
ARM directives
add r2, pc, #1
bx r2
THUMB [...]

BRANCH & LINK (BL)

SYNTAX

bl[cond] label
bl label

LR <- PC
LR = 0x10060

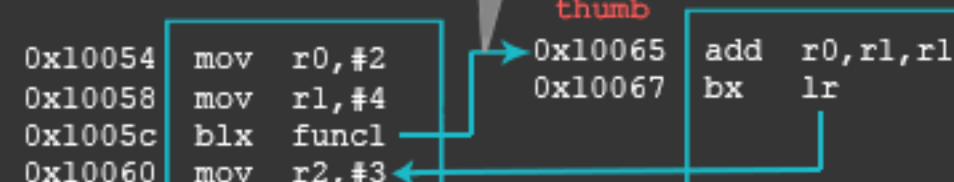


BRANCH & LINK & EXCHANGE (BLX)

SYNTAX

blx[cond] Rm
blx Rm
blx label

LR <- PC
LR = 0x10060



Beispiel

...

```
MOV32    R8,#COUNTER    ;Zähler initialisieren
MOV      R4,#0           ;Vergleichswert laden
BL       Toggle          ;Unterprogramm aufrufen
```

Loop

```
;Zählschleife für Delay
CMP      R8,R4           ;Vergleiche R8 mit R4
BEQ      Toggle          ;Wenn CMP Flag gesetzt Sprung auf Toggle
SBC      R8,#1           ;Subtraktion mit Carry
B        Loop            ;Sprungbefehl zum Label Loop
```

Toggle

```
LDR      R1,=GPIO_PORTA_DATA_R ;Lade Konstante in R1
EOR      R0,#PA7           ;XOR
STR      R0,[R1]           ;Speicher an Adresse von R1
MOV32    R8,#COUNTER       ;Zähler initialisieren
BX       LR                ;Rücksprung zum Anfang
```

...

Beispiel

	Adresse	Maschinencode	Assembler-Code
14	00000000		THUMB
15	00000000		AREA .text ,CODE,READONLY,ALIGN=2
16	00000000		EXPORT Blinky
17	00000000		ENTRY
18	00000000		
19	00000000		;Funktionen
20	00000000		Blinky
21	00000000		;Als erstes Clock auf Port A aktivieren
22	00000000	4912	LDR R1,=SYSCTL_RCGC2_R
23	00000002	6808	LDR R0,[R1]
→ 24	00000004	F040 0001	ORR R0,R0,#SYSCTL_RCGC2_GPIOA
25	00000008	6008	STR R0,[R1]
26	0000000A		

F040 0001 ORR R0,R0,#SYSCTL_RCGC2_GPIOA

Beispiel

AND R3, R1, R2
ORR R4, R1, R2
EOR R5, R1, R2

		Source Register			
R1		0100 0110	1010 0001	1111 0001	1011 0111
R2		1111 1111	1111 1111	0000 0000	0000 0000
		Result			
R3		0100 0110	1010 0001	0000 0000	0000 0000
R4		1111 1111	1111 1111	1111 0001	1011 0111
R5		1011 1001	0101 1110	1111 0001	1011 0111

LSL R0, R5, #7
LSR R1, R5, #17
ASR R2, R5, #3
ROR R3, R5, #21

		Source Register			
R5		1111 1111	0001 1100	0001 0000	1110 0111
		Result			
R0		1000 1110	0000 1000	0111 0011	1000 000
R1		0000 0000	0000 0000	0111 1111	1000 1110
R2		1111 1111	1110 0011	1000 0010	0001 1100
R3		1110 0000	1000 0111	0011 1111	1111 1000

Beispiel fürs Branching

```
MOV R2, #17      ; R2 = 17
B  TARGET        ; branch to target
ORR R1, R1, #0x4  ; not executed

TARGET
  ADD R1, R1, #78  ; R1 = R1 + 78
```

 Bezeichnungen (engl. Labels) wie z.B. TARGET zeigen den Ort der Anweisung an. Labels können keine reservierten Wörter sein (wie ADD, ORR, ect)

Thank
You

